

Project Title: Secure Language Assembly Inspector Tool (SLAIT)

Team Members:

- Maria Linkins-Nielsen (mlinkinsniel2022@my.fit.edu)
- Michael Bratcher (mbratcher2021@my.fit.edu)

Faculty Advisor: Dr. Marius Silaghi (msilaghi@fit.edu)

Client: Dr. Marius Silaghi CSE Professor & Faculty Advisor

Planning Meeting Dates with Client:

- 01/16/26

Goal and Motivation: Develop a secure, streamlined way to inspect snippets of code at a low level of abstraction.

Students within the CSE 3120 “Computer Architecture and Assembly” class are required to place their systems into a vulnerable state to perform tasks involving executing “low-level language” based code (LLLC), such as those running x86 and Java Bytecode. This can leave the system at risk of unintentional damage by student actors from faulty lines of LLLC that would usually be stopped by the system, or by bad actors taking advantage of a system placed in such a state. This issue does not just affect these students, but anybody attempting to optimize their code by inspecting the assembly, as observing and running assembly to inspect registers is usually a tedious task that involves placing your PC in a vulnerable state. “SLAIT” attempts to solve this problem by code requested, allowing the user to specify registers and flags at specific locations in the code, compiling it within a Docker environment, and returning the requested information to the user, all without the user being required to lower security levels within their device.

Approach (key features of the system):

- Inspect your code securely, without risking your system. Submit your low-level code snippets directly into SLAIT, choose between x86, and analyze them without ever putting your device into a vulnerable state.
- Track registers and flags at the moments that matter. Mark the exact points in your code where you want to observe registers or flags. SLAIT automatically records these registers and flag states at your desired points, so you get the insights you need.
- Run your code in a safe backend environment. Your snippets are executed inside a Docker-based sandbox. That means no matter what your code does, it stays isolated and cannot damage your personal machine.

- See clear results and compare runs easily. View an organized timeline of how your registers and flags change over execution. Compare different runs with varied initial states, spot errors you didn't expect, and measure runtime details, all presented in a way that helps you make faster decisions.

Novel features/functionalities:

- Most low-level inspection tools either require unsafe local execution or lengthy configuration of VMs/sandboxes. SLAIT provides secure, on-demand inspection in a streamlined way. The combination of Docker isolation, fine-grained register/flag tracking, and comparative visualization in a user-friendly Angular frontend makes SLAIT unique compared to existing tools, which usually do not come equipped with as much visual utility in reference to flag and register observation.

Algorithms and tools: potentially useful algorithms and software tools

- Docker
 - Sandbox to execute code within
 - Easy to create new instances
 - Low overhead in comparison to a VM
- Angular
 - Effective tool for front-end development
 - Useful for splitting independent features into their own components
- Compiler toolchains
 - NASM + GCC (assembler and linker for x86-64 ELF binaries)
 - Used to assemble and link user-provided assembly code before execution
- Debugging Tools
 - GDB (GNU Debugger)
 - Used to set breakpoints and capture register and flag state during program execution
- Parsing libraries
 - Parsing frontend-provided configuration files to generate automated GDB breakpoint scripts
- Visualization libraries
 - (D3.js, Angular charting libraries) - helpful for timeline/register visualization

Technical Challenges: Discuss three main CSE-related challenges.

- Limited prior experience designing REST-based backend APIs and request-handling pipelines.
- We plan to utilize a backend server, but we have not had much experience with frontend backend development.
- Ensuring correct parsing/instrumentation of user code for register/flag observation.

Design

- Single-user web-based application interface
- Web browser-based frontend developed using Angular
- Unified web and backend server hosting frontend assets and execution API
- Docker container sandbox used for isolated code execution and debugging
- REST-based HTTP communication between frontend and backend
- Possible (not in diagram) session database for user session tracking and future authentication support
 - Idea from Dr. Silaghi

Evaluation

- Speed Tests:
 - Time to return output to the user after a test has been requested?
 - Time for a user to make a complete test from scratch?
- Accuracy Tests:
 - Percentage of accurate flag changes returned?
- User Surveys in later milestones.

Progress Summary

Module/Feature	Completion %	To do
Frontend Input Features	85%	Templates, Multi Language Selection
Frontend Return Features	0%	Format Returned Data, Graphs
Backend Execution Pipeline	70%	HTTP Integration, container automation trigger
Register Tracking (GDB Integration)	90%	JSON Formatting for frontend
Docker Sandbox Environment	80%	Final image cleanup and deployment optimization
Backend API Integration	30%	Multipart file upload handling and routing

Milestone 4 (Feb 23): itemized tasks:

- Frontend Output Widget
- Finalize Backend Docker Execution and Register Tracking Pipeline
- Add Frontend Templates
- Implement Multipart HTTP File Upload and Backend API Integration

Milestone 5 (Mar 30): itemized tasks:

- Integrate full frontend-backend execution pipeline
- Testing of full frontend-backend execution pipeline
- Further testing as needed
- Create poster for Senior Design Showcase

Milestone 6 (Apr 20): itemized tasks:

- Test/demo entire system
- Conduct evaluation and analyze results
- Create user/developer manual
- Create demo video

Task Matrix for Milestone 4:

Task	Michael	Maria
Frontend Output Widget	100%	0%

Finalize Backend Docker Execution & Register Tracking Pipeline	0%	100%
Add Frontend Templates	100%	0%
Implement Backend API & File Upload	25%	75%

Approval from Faculty Advisor:

- I have discussed with the team and approved this project plan. I will evaluate the progress and assign a grade for each of the three milestones.
- Signature: _____ Date: _____